ENGENHARIA WEB · UTAD · 2026/2027

Guia de Estudo Teste Teórico 1

Toda a matéria das Aulas 01 a 06, explicada do zero, com mnemónicas para decorar, esquemas e as armadilhas do gabarito do professor.

`HTTP/1.1 200 OK`

É assim que queres que corra o teste.

1 Aplicações cliente-servidor
2 URL e navegação numa página web
3 HTTP: terminologia e mensagens
4 Métodos, GET vs POST
5 Cabeçalhos (headers)
6 Tipos MIME
7 Códigos de estado
8 Servidor: resolução de endereços
9 Conteúdo estático e dinâmico
10 HTML visto pelo cliente
11 Proxies e tipos de cache
12 Autenticação e cookies
13 Armadilhas do gabarito
14 Cábula final

Fontes: slides das Aulas 01 a 04 e da Aula sobre conteúdos estáticos e dinâmicos; Shklar & Rosen, *Web Application Architecture*, cap. 3 e cap. 6; perguntas reais do Moodle de anos anteriores.

CAPÍTULO 1

Aplicações cliente-servidor

A ideia base de tudo o resto: alguém pede, alguém responde.

O que é

Numa aplicação **cliente-servidor** há dois papéis. O **cliente** inicia sempre a comunicação com um **pedido**. O **servidor** fica à espera, processa o pedido e devolve uma **resposta**. Os dois têm de falar o mesmo **protocolo** (as regras da conversa) e normalmente estão em máquinas diferentes, ligados pela rede.

MNEMÓNICA · QUEM FALA PRIMEIRO

O cliente é o freguês, o servidor é o empregado de mesa.

O empregado nunca vai à mesa pedir-te nada: só responde ao que pedes. Por isso, **o servidor não faz pedidos ao cliente** (pergunta real do Moodle).

Portas mais comuns

PORTA	PROTOCOLO	PORTA	PROTOCOLO
`20 / 21`	FTP (dados / controlo)	`80`	**HTTP**
`22`	SSH (acesso seguro)	`110`	POP3 (receber e-mail)
`23`	Telnet (texto sem cifra)	`143`	IMAP4 (receber e-mail)
`25`	SMTP (enviar e-mail entre servidores)	`443`	**HTTPS** (HTTP sobre TLS/SSL)
`53`	DNS (nomes para IPs)	`993 / 995`	IMAP4 / POP3 sobre SSL

MNEMÓNICA · PORTAS

«Oitenta é a porta da rua, 443 é a porta blindada.» HTTP 80, HTTPS 443.
FTP 20-21: «vinte e um anos, já pode transferir». **SSH 22** vem logo a seguir, **Telnet 23** é o irmão inseguro do SSH. **SMTP 25:** «o correio sai ao dia 25». **DNS 53.** Receber correio: **POP 110** (é o mais velho, número redondo) e **IMAP 143**.

Aplicação web vs aplicação desktop

Uma **aplicação Web** é um programa **guardado num servidor remoto** e **entregue pela Internet através da interface de um navegador**. Uma aplicação desktop (standalone) está instalada na máquina: código, dados locais e base de dados (integrada ou num servidor) correm sobre o sistema operativo local.

Na versão web de um CRM, o código fica dividido: uma parte corre no **browser** (cliente) e outra no **servidor web**, junto à base de dados. O que circula entre os dois são **pedidos e respostas HTTP**.

PRÓS DA WEB	CONTRAS DA WEB
Multiplataforma	Segurança
Atualizações automáticas	Precisa de Internet
Instalação dispensada	
Poupa recursos da máquina	

PRÓS DO DESKTOP	CONTRAS DO DESKTOP
Desempenho	Preso ao dispositivo
Offline (sem Internet)	Gasta disco
Segurança (mais controlo)	Instalação e atualizações manuais

MNEMÓNICA · VANTAGENS

A Web dá-te MAIS, o Desktop corre em DOS.

MAIS (Web): **M**ultiplataforma · **A**tualizações automáticas · **I**nstalação dispensada · **S**em gastar recursos.
DOS (Desktop): **D**esempenho · **O**ffline · **S**egurança controlada.

Pergunta real. «Uma aplicação Web para ser usada necessita de:» → **um servidor e um cliente**. «Apenas um servidor» está errado.

CAPÍTULO 2

URL e navegação numa página web

O que acontece entre escreveres um endereço e veres a página.

Anatomia de um URL

```
scheme:[//[user:password@]host[:port]][path][?query][#anchor]

https://www.continente.pt/pt-pt/public/Pages/searchresults.aspx?k=arroz
└──┘   └──────────┘└──────────────────────────────────────────┘ └──┘
esquema       host                    path                  query string
```

- **Esquemas**: http, https, ftp, mailto, file, data, irc.
- A **query string** (depois do `?`) leva pares `nome=valor` separados por `&`.
- A **âncora** (`#...`) aponta para uma parte da página.
- Os URLs só podem usar caracteres **ASCII**. O **URL encoding** troca cada carácter inseguro por `%` seguido de 2 dígitos hexadecimais, e o **espaço** passa a `%20` ou `+`. Exemplo: `Malásia` → `Mal%C3%A1sia` (UTF-8).

MNEMÓNICA · ORDEM DO URL

«Eu Uso Senha Num Porto, Passo Pela Query, Até à Âncora»

Esquema · **U**tilizador:**S**enha@ · **N**ome do host · **P**orta · **P**ath · ?**Q**uery · #**Â**ncora.

Sequência pedido-resposta de uma página

1. Utilizador escreve o URL, clica num link ou num favorito → **2. Pedido HTTP** o browser pede a página ao servidor → **3. Servidor** abre o ficheiro (estático) ou executa código (dinâmico) → **4. Browser** mostra o HTML recebido

5. Pedidos extra um por cada imagem, CSS e script (no exemplo dos slides foram 35), possivelmente a outros servidores → **6. Mostra** vai mostrando cada elemento à medida que as respostas chegam → **7. JavaScript** executa o código do cliente, que pode gerar ainda mais pedidos

Porque é que o browser faz pedidos extra? Porque o HTML só tem *referências* às imagens, CSS e scripts, não os próprios ficheiros. Cada referência obriga a um novo pedido. Conta assim: 1 HTML + 10 imagens + 2 CSS + 3 JS = **16 pedidos**.

Estático vs dinâmico, visto pelo browser

Conteúdo estático: o servidor abre o ficheiro (por exemplo `a-diocese.html`) e devolve-o. **Conteúdo dinâmico**: o servidor **executa o código** associado ao recurso e o resultado é a resposta, que pode ser HTML, outro código ou até binário.

MNEMÓNICA · O BROWSER É «CEGO» À COZINHA

Ao cliente chega o prato, não a receita.

Seja comida de lata (estático) ou feita na hora (dinâmico), o browser recebe sempre uma resposta HTTP. Pergunta real: receber conteúdo dinâmico **«é o mesmo que receber um conteúdo estático»**.

CAPÍTULO 3

HTTP: terminologia e mensagens

HyperText Transfer Protocol: as palavras que o professor usa e a forma exata das mensagens.

Definições a saber de cor

- **HTTP** é um protocolo da **camada de aplicação** que usa **TCP** (porta 80) como transporte. Segue o paradigma **pedido-resposta**.
- Resposta do gabarito à pergunta «HTTP é:» → **«um protocolo de comunicação usado por todos os agentes web num canal de comunicação»**. A opção «um protocolo de comunicação», sozinha, só dá meia cotação.
- **Agentes**: cliente HTTP (por exemplo o browser), servidor HTTP (Apache, IIS, Kestrel) e, pelo meio, **proxies**.
- **Proxy**: intermediário que é **servidor para o cliente e cliente para o servidor**. Serve para **cache**, **balanceamento de carga**, **filtragem de conteúdos** e segurança, e permite atravessar firewalls. Pode haver vários proxies em cadeia.
- **Gateway**: intermediário que passa a mensagem a outro sistema (é daí que vem o «502 Bad Gateway»).
- **Ligação**: o «circuito virtual» entre browser, proxies e servidor.

Stateless: sem estado

Cada troca HTTP é **um pedido e uma resposta**, e o servidor **não se lembra** de nada entre pedidos. Protocolos como FTP, SMTP e POP são *stateful*, porque mantêm uma sessão. No HTTP/1.1 a ligação pode manter-se aberta (**persistent connection**), mas isso serve só para eficiência e **não guarda estado**. Para haver estado é preciso construí-lo por cima, com cookies e sessões.

MNEMÓNICA · STATELESS

O HTTP tem memória de peixinho dourado.

Cada pedido é um desconhecido novo. Os cookies são o post-it que o cliente cola na testa para o servidor o reconhecer.

A estrutura das mensagens

`GET /a-diocese.html HTTP/1.1`	linha de pedido

`Host: www.diocese-vilareal.pt`
`User-Agent: Mozilla/5.0 ...`
`Referer: http://home.utad.pt/...`
	cabeçalhos

`(linha vazia: CRLF)`	separador
`(corpo: vazio num GET)`	corpo

`HTTP/1.1 200 OK`	linha de estado

`Content-Type: text/html`
`Content-Length: 9934`
`Server: Microsoft-IIS/4.0`
	cabeçalhos

`(linha vazia: CRLF)`	separador
`<HTML> ...`	corpo

LINHA	SINTAXE	EXEMPLO
Linha de pedido	`método URI-do-pedido versão-HTTP`	`GET /tc/home.html HTTP/1.1`
Linha de estado	`versão-HTTP código frase-explicativa`	`HTTP/1.1 404 Not Found`

MNEMÓNICA · AS PRIMEIRAS LINHAS

Pedido = «MUV»: **M**étodo · **U**RI · **V**ersão. «O cliente **MUV**e-se para pedir.»
Resposta = «VCF»: **V**ersão · **C**ódigo · **F**rase. «**V**ê **C**omo **F**icou.»
Mensagem = carta: linha de topo (a quem e o quê) · carimbos (cabeçalhos) · linha em branco · conteúdo (corpo).

Armadilha (pergunta real): ao escrever `https://www.ads.pt/` na barra, a linha gerada é **`GET / HTTP/1.1`**. Não é `HTTPS/1.1`, porque a versão é sempre «HTTP». Também não leva o domínio, que vai no cabeçalho `Host`, nem o URL completo.

A **frase explicativa** (OK, Not Found…) é para humanos: mudá-la não altera o comportamento de um cliente bem feito, que só olha para o código.

CAPÍTULO 4

Métodos HTTP e GET vs POST

Os verbos do protocolo e a diferença que sai sempre em teste.

VERSÃO	MÉTODOS INTRODUZIDOS	PARA QUÊ
HTTP 0.9	**GET**	Pedir um recurso (página, imagem…)
HTTP 1.0	**HEAD**, **POST**	HEAD: só os cabeçalhos, sem corpo (para comparar versões). POST: enviar dados no corpo (formulários).
HTTP 1.1	PUT, DELETE, OPTIONS, TRACE, CONNECT, PATCH	PUT: guardar um recurso no servidor (upload). DELETE: apagar…

MNEMÓNICA · VERSÕES

0.9 → G. 1.0 → G + H + P («**GHP**, Grande Hotel do Porto»). **1.1 → o resto**.

Os sites e aplicações web normais usam **só GET e POST**. As **APIs REST** usam os métodos como operações CRUD:

MÉTODO	CRUD	MNEMÓNICA
POST	Create	«**P**ostar é **p**ublicar algo novo»
GET	Read	«**G**et = **G**anhar, trazer cá»
PUT	Update / Replace	«**Put**: pôr no lugar, substituir tudo»
PATCH	Partial update	«**Patch** = remendo: só um bocado»
DELETE	Delete	«apagar»

GET vs POST

```
<form action="/pesquisa" method="get">  →  GET /pesquisa?q=UTAD HTTP/1.1      (dados no URL)
<form action="/pesquisa" method="post"> →  POST /pesquisa HTTP/1.1  ...  q=UTAD  (dados no
corpo)
```

	GET	POST
Onde vão os dados	No URL (query string), à vista de todos	No corpo da mensagem
Voltar atrás / recarregar	Inofensivo	Reenvia os dados (o browser avisa)
Favoritos / cache / histórico	Sim / sim / ficam no histórico	Não / não / não ficam
Tamanho / tipo de dados	Limitado (URL até cerca de 2048 caracteres), só ASCII	Sem limites, aceita binário
Segurança	Menos segura: **nunca para passwords**	Um pouco mais segura (não fica em logs nem no histórico)

MNEMÓNICA · GET VS POST

GET é um postal, POST é uma carta fechada.

O postal toda a gente lê (URL visível), cabe pouco (tamanho limitado), podes fotocopiar e guardar (favoritos, cache, histórico) e só leva texto (ASCII). A carta fechada leva o que quiseres lá dentro (corpo, binário, ficheiros), mas não a podes guardar nos favoritos.

Exame de 2021. Para uma pesquisa tipo Google, o melhor é **GET**: a pesquisa não altera dados e o URL com o resultado pode ser guardado e partilhado.

CAPÍTULO 5

Cabeçalhos HTTP

Metadados em pares `nome: valor`, nos pedidos e nas respostas.

Os cabeçalhos deixam cliente e servidor passar **informação adicional**. Ficam depois da primeira linha e antes da linha vazia, e **usam-se nos pedidos e nas respostas** (pergunta real).

Os 4 grupos

GRUPO	O QUE FAZ	EXEMPLOS
General	Aplica-se a pedidos e respostas, sem descrever o corpo	`Date`, `Connection`, `Cache-Control`, `Warning`
Request	Informação sobre o pedido ou sobre o cliente	`Host`, `User-Agent`, `Referer`, `Authorization`, `Accept-Encoding`, `If-Modified-Since`
Response	Informação sobre a resposta ou sobre o servidor	`Location`, `Server`, `WWW-Authenticate`
Entity	Descreve o corpo (o conteúdo)	`Content-Type`, `Content-Length`, `Last-Modified`, `Content-Encoding`

MNEMÓNICA · GRUPOS

«Gatos Raramente Respondem Educadamente»

General · **R**equest · **R**esponse · **E**ntity. E a regra de ouro: se começa por **Content-**, fala do **corpo**, logo é **Entity**.

Os cabeçalhos que mais saem

HEADER	TIPO	PARA QUE SERVE
`Host`	Pedido	Nome do servidor de destino. Permite o **virtual hosting** (vários sites num servidor). Obrigatório no HTTP/1.1.
`User-Agent`	Pedido	Identifica o software cliente (browser). O servidor pode diferenciar a resposta por cliente.
`Referer`	Pedido	URL da página onde estava o link que originou o pedido. Não é enviado se o URL foi escrito à mão.
`Authorization`	Pedido	Credenciais, enviadas depois de um 401.
`Accept-Encoding`	Pedido	O cliente diz que **aceita conteúdo comprimido** e com que método (`gzip`, `x-zip`…).
`If-Modified-Since`	Pedido	Pedido condicional: «manda só se mudou desde esta data», senão o servidor responde **304**.
`Location`	Resposta	Novo URL nos redirecionamentos **301/302**.
`WWW-Authenticate`	Resposta	Acompanha o **401** e diz o «realm» protegido.
`Server`	Resposta	Software do servidor (por exemplo Microsoft-IIS ou Kestrel).
`Content-Type`	Entidade	Tipo **MIME** do corpo. Aparece em respostas e também em pedidos com corpo (POST).
`Content-Length`	Entidade	Tamanho do corpo em **bytes**. Obrigatório em pedidos com corpo (sem ele: 411).
`Last-Modified`	Entidade	Data da última alteração do conteúdo. Essencial para a cache.
`Date`	Geral	Quando a mensagem foi criada.

Referer (gabarito com 4 certas): indica o URL da página que estava a ser mostrada · o cliente indica o local que deu origem ao pedido · permite referenciar um recurso que não está no servidor de destino · permite distinguir um clique num link de um URL escrito na barra. **Errada:** «o servidor indica a referência base da resposta» (o Referer é um header de pedido, não de resposta). Nota: *Referer* está mal escrito de propósito na norma, só com um «r» antes do «er».

> **Accept-Encoding (gabarito com 2 certas):** «indica que o cliente aceita conteúdos comprimidos» **e** «indica qual o método de compressão que pode ser usado na resposta». Não tem nada a ver com cifra nem com segurança.

Quem insere os cabeçalhos?

- **Pedidos**: o browser e também o código da aplicação (por exemplo, JavaScript com `request.setRequestHeader("Content-Type", ...)`).
- **Respostas a conteúdo estático**: o servidor web.
- **Respostas a conteúdo dinâmico**: o servidor web e também o código da aplicação (`Response.AddHeader("CustomHeader","CustomValue")`).

Response buffering: o servidor retém a resposta até o script acabar (ou até `Flush` / `End`). É por isso que um `AddHeader` escrito a meio do HTML ainda funciona, já que os cabeçalhos ainda não saíram.

Forbidden header names

São cabeçalhos que o código **não pode alterar**, porque o browser controla-os: os que começam por `Proxy-` ou `Sec-`, e `Accept-Charset`, `Accept-Encoding`, `Connection`, `Content-Length`, `Cookie`, `Date`, `Host`, `Origin`, `Keep-Alive`, `Referer`…

MNEMÓNICA · PROIBIDOS

O browser não deixa mexer em «quem sou, por onde vou e o que aceito».

Quem sou: Host, Origin, Referer, Cookie. **Por onde vou**: Connection, Keep-Alive, Content-Length, Date. **O que aceito**: Accept-Charset, Accept-Encoding. Mais tudo o que começa por **Proxy-** e **Sec-**.

CAPÍTULO 6

Tipos MIME

Multipurpose Internet Mail Extensions: a etiqueta que diz o que vem no corpo.

O MIME nasceu no **e-mail** e foi adotado pelo HTTP. O browser não confia na extensão do ficheiro: decide o que fazer com o corpo (mostrar HTML, desenhar uma imagem, chamar um helper…) pelo **Content-Type**.

```
Content-Type: tipo/subtipo [; parâmetros]

text/html        text/plain       text/css
image/gif        image/jpeg       audio/x-wav     video/mpeg-2
text/html; charset=utf-8          (o charset por omissão é ISO-8859-1)
```

MNEMÓNICA · FORMATO MIME

«Família / Pessoa ; Detalhe»

`image/gif` = família *image*, membro *gif*. Depois do **;** vêm os detalhes, como o charset. E o charset por omissão é **ISO-8859-1**: «o **I**SO é o **I**nicial».

image/gif (gabarito com 2 certas): «a mensagem tem no corpo os bytes de uma imagem» **e** «o cliente recebeu uma imagem».

Casos especiais

- **multipart**: várias partes num corpo, separadas por uma fronteira (`boundary`):

  ```
  Content-Type: multipart/x-mixed-replace; boundary=XPTOText
  --XPTOText
  Content-Type: image/gif
  ...
  --XPTOText--          ← dois hífenes no fim = acabou
  ```

- **Base64 / data URI**: incorpora o conteúdo no próprio HTML, **sem pedido HTTP extra**: `<img src="...">`, com o formato `data:[<mediatype>][;base64],<dados>`.
- A **compressão** negoceia-se à parte: o cliente envia `Accept-Encoding` e o servidor indica no `Content-Encoding` o que usou.
- **Pode ser alterado por código?** Sim: numa resposta dinâmica a aplicação define o Content-Type (é assim que um controller devolve JSON ou um ficheiro PDF).

CAPÍTULO 7

Códigos de estado

Três dígitos na linha de estado que classificam a resposta.

Porque existem? (Gabarito com 2 certas:) **para classificar a resposta com um tipo e contexto** e **para os clientes interpretarem de forma fácil e rápida o tipo de resposta que recebem**. Não são um checksum.

CLASSE	SIGNIFICADO	MNEMÓNICA
1xx	Informação: pedido recebido, a continuar	«**Aguenta aí**»
2xx	Sucesso: recebido, entendido e aceite	«**Aqui tens**»
3xx	Redireção: é preciso mais uma ação	«**Vai ali**»
4xx	Erro do cliente: pedido mal feito ou impossível	«**A culpa é tua**»
5xx	Erro do servidor: falhou um pedido aparentemente válido	«**A culpa é minha**»

MNEMÓNICA · CLASSES

«Aguenta · Toma · Vai · Tu · Eu»

1 aguenta, 2 toma, 3 vai, 4 a culpa é **tua**, 5 a culpa é **minha (servidor)**.

Os códigos a saber

CÓDIGO	NOME	QUANDO E COMO LEMBRAR
100	Continue	«Podes mandar o resto do corpo» (o cliente envia `Expect: 100-continue`).
200	OK	Tudo certo, o recurso vem no corpo.
201	Created	Foi criado um recurso novo.
206	Partial Content	Vem só um pedaço (downloads em partes).
301	Moved Permanently	Mudou-se **para sempre**, com `Location`. Proxies e browsers podem guardar em cache.
302	Found	Mudou-se **temporariamente**, com `Location`. É o que o `RedirectToAction` devolve.
303	See Other	«Vê noutro sítio, com GET» (depois de um POST).
304	Not Modified	«Usa a tua cópia em cache», em resposta a um `If-Modified-Since`. Vai sem corpo.
307 / 308	Temporary / Permanent Redirect	Como o 302 / 301, mas **mantêm o método** (um POST continua POST).
400	Bad Request	Pedido mal formulado (sintaxe errada).
401	Unauthorized	«**Quem és tu?**»: falta autenticação. Vai com `WWW-Authenticate`.
403	Forbidden	«**Sei quem és, mas não entras.**»
404	Not Found	«**Não existe.**»
411	Length Required	Falta o `Content-Length`.
500	Internal Server Error	O código do servidor rebentou.
502	Bad Gateway	O **intermediário** (proxy ou gateway) recebeu uma resposta inválida do servidor de cima (upstream).
503	Service Unavailable	Servidor em baixo ou sobrecarregado.
505	HTTP Version Not Supported	Versão do HTTP não suportada.

MNEMÓNICAS · OS PARES QUE CONFUNDEM

301 vs 302: «**1** = **1ª** e única morada nova (permanente); **2** = **2ª** casa, de férias (temporário).»
401 vs 403: «401: o porteiro pede-te o cartão. 403: viu o cartão e mesmo assim não entras.»
502: «**Bad Gateway** = o estafeta (proxy) foi buscar a encomenda e trouxe lixo.»

502 (gabarito com 2 certas): «o servidor proxy que representa o servidor de gateway não consegue obter um bom resultado do servidor de upstream» **e** «problemas de acesso provocados por uma falha na comunicação entre dois ou mais servidores que retransmitem dados entre si».

404, a maior armadilha: o gabarito do professor deu como certas «**erro de processamento do pedido**» e «**erro por pedido mal formulado**». Na norma, o 404 é *Not Found* e «mal formulado» é o 400. Se te aparecer esta pergunta com estas opções, responde como o gabarito.

Exemplos dos slides

```
http://www.utad.pt   →  307 → http://www.utad.pt/  → 200
http://www.utad.pt/pt/index.asp  →  302 Object Moved, Location: .../pt/index.html
ASP.NET:  HTTP/1.1 302 Found · Content-Length: 0 · Location: /MyController/MyRoute
```

Diretoria sem a barra final (livro): `/~shklar` é redirecionado (301) para `/~shklar/`. Sem a barra, os URLs relativos da página seriam resolvidos de forma errada.

CAPÍTULO 8

Servidor web: resolução de endereços

Como o servidor descobre que ficheiro corresponde a `/tc/index.html` e se o podes ver.

O caminho de um pedido no servidor

Suporte de comunicações de rede
recebe o pedido e envia a resposta

→

1. Resolução de endereços
Virtual hosting · Address mapping · Pseudónimos · Autenticação

→

2. Processamento do pedido
estático (static, as-is) ou dinâmico (CGI, SSI, APIs, Servlets…)

→

3. Geração da resposta

MNEMÓNICA · AS 3 ETAPAS

O servidor joga RPG: Resolve → Processa → Gera.

Definição do gabarito: a secção «Address Resolution» é **«um conjunto de etapas que visam identificar os ficheiros responsáveis pela produção da resposta e verificar a permissão de acesso a estes»**.

MNEMÓNICA · OS 4 PASSOS DA RESOLUÇÃO

«Vê A Pasta Autorizada»

Virtual hosting (que site?) → **A**ddress mapping (que pasta e ficheiro?) → **P**seudónimos (há atalhos?) → **A**utenticação (tens autorização?). A frase resume o que a resolução faz: encontrar a pasta e verificar se estás autorizado.

1. Virtual hosting

Um servidor (e até um único IP) aloja **vários sites**. Como sabe qual queres? Pelo cabeçalho `Host`. Sem o Host, `GET /tc/index.html` enviado para `188.114.97.5` seria ambíguo.

```
<VirtualHost *:80>
    ServerName   www.tribunalconstitucional.pt
    DocumentRoot /var/www
    <Directory /var/www>  DirectoryIndex index.php ... </Directory>
</VirtualHost>
```

> **Exame de 2021.** Um servidor com várias aplicações, todas com `wwwroot\images\logo.png`, recebe `GET /images/logo.png`. Como decide qual usar? Pelo **Host**, que identifica o virtual host, e pela raiz (DocumentRoot) desse host.

2. Address mapping

Traduz o caminho do URL para um ficheiro no disco, a partir da **raiz web** (DocumentRoot). Não faz sentido a raiz do URL ser a raiz do disco, porque exporia o sistema todo.

SERVIDOR	RAIZ WEB POR OMISSÃO
Apache (Linux)	`/var/www`
Apache (Windows)	`C:/Program Files/Apache Software Foundation/Apache2.2/htdocs/`
IIS (Windows)	`c:\Inetpub\wwwroot`
ASP.NET Core	a pasta `wwwroot` do projeto

Exemplo: com `DocumentRoot /var/www`, `/tc/index.html` → `/var/www/tc/index.html`. O servidor verifica se a pasta `tc` existe e se tem lá o ficheiro `index.html`.

3. Pseudónimos (aliases)

```
Alias "/image" "/ftp/pub/image"          → /image/x.png vem de /ftp/pub/image/x.png
Alias /tc/ /var/tc/
Alias /tc/index.html /site/index.asp?id=tribunal
```

Ligam um caminho do URL a uma pasta **fora** da árvore do DocumentRoot. Assim, **dão acesso a conteúdo que o virtual host normalmente não alcançaria** e permitem **partilhar ficheiros entre virtual hosts**.

Pseudónimos (gabarito com 3 certas): **mascarar caminhos para ficheiros · mudar os nomes aos ficheiros · associar recursos externos à estrutura de ficheiros da aplicação web**. Erradas: criar endereços do servidor, esconder ficheiros, proteger de ataques.

MNEMÓNICA · PSEUDÓNIMO

Um pseudónimo é um nome artístico.

O ficheiro tem o nome verdadeiro e a morada verdadeira no disco, mas o público conhece-o pelo nome artístico (o caminho no URL). **Mascara** o caminho, **muda o nome** e **traz de fora** conteúdo para a aplicação.

4. Autenticação

```
Servidor:  HTTP/1.1 401 Unauthorized
           WWW-Authenticate: Basic realm="User Visible Realm"
Cliente:   GET /tc/index.html HTTP/1.1
           Authorization: Basic QWxhZGRpbjpPcGVuU2VzYW1l    (user:pass em Base64)
```

Módulo de Autenticação (gabarito com 2 certas): **«a componente que verifica se o pedido tem as permissões de acesso ao recurso solicitado»** e **«a secção responsável por gerar mensagens de resposta com o código 401»**. Erradas: «verifica username e password», «gera um formulário», «onde os utilizadores se autenticam».

No fim da resolução de endereços já se sabe o **recurso (ficheiro)** e a **autenticação está verificada**. Falta saber se o ficheiro é estático ou dinâmico, e isso é o capítulo seguinte.

CAPÍTULO 9

Entrega de conteúdo estático e dinâmico

Depois de encontrar o ficheiro: devolvê-lo tal como está ou executá-lo?

Conteúdo estático

TIPO	COMO FUNCIONA
Static content pages	O servidor **gera os cabeçalhos** e junta o ficheiro como **corpo**.
Páginas «as-is»	O ficheiro **já é a mensagem HTTP completa**, com os cabeçalhos incluídos, e é enviado tal e qual.

MNEMÓNICA · STATIC VS AS-IS

«Static: o servidor põe o envelope. As-is: a carta já vem no envelope.»

Conteúdos estáticos (gabarito com 3 certas): são assim chamados **por terem um conteúdo estático · porque o seu conteúdo corresponde integralmente ao conteúdo da mensagem de resposta · porque obtê-los não implica processar código de qualquer espécie**. Errada: «por estarem sempre na mesma pasta».

Conteúdo dinâmico: as tecnologias

TECNOLOGIA	IDEIA-CHAVE
CGI, Common Gateway Interface	O servidor percebe que o pedido é para um programa (por exemplo Perl) e não para um HTML, e **cria um processo filho por pedido**. O programa lê as variáveis CGI e devolve um documento. Problema: **um processo novo por pedido não escala**.
SSI, Server Side Includes	Fragmentos dinâmicos (outros ficheiros, datas, resultados de comandos) inseridos **dentro do HTML** através de **macros**, sem o peso de um CGI completo.
APIs nativas	Código **compilado** para um servidor específico: **Apache Server API**, **ISAPI** (IIS).
FastCGI	Resolve o problema do CGI: **processos persistentes** que ficam vivos entre pedidos.
Template processing	HTML com código lá dentro: **PHP**, **Cold Fusion**, **Active Server Pages**.
Servlets / JSP	As soluções Java: Servlets e Java Server Pages.

MNEMÓNICA · TECNOLOGIAS DINÂMICAS

«Cada Servidor Ainda Faz Truques Super Java»

CGI · **S**SI · **A**PIs nativas · **F**astCGI · **T**emplates · **S**ervlets · **J**SP.

CGI vs FastCGI: «O CGI chama um **táxi novo para cada passageiro**. O FastCGI tem um **autocarro sempre a circular**.»

Passos do servidor com um CGI

1. Determina se é um programa CGI.
2. Traduz para o ficheiro correspondente.
3. Verifica se é um ficheiro válido.
4. Verifica as permissões.
5. Define as variáveis de ambiente.
6. Cria um processo filho para correr o programa.
7. Processa a resposta do programa CGI.

Perguntas reais. CGI → **«serve para produzir dinamicamente conteúdos de respostas»** (não é «mais rápido» nem «reencaminhar para outro servidor»). «As linguagens como PHP, Java, Perl…» → **«nenhuma das opções apresentadas»**, porque as opções eram todas erradas: não são usadas pelo cliente, não servem «unicamente» para extensões, não produzem conteúdo estático e não armazenam conteúdo.

CAPÍTULO 10

HTML visto pelo cliente

O que o browser faz com o HTML que recebe: os pedidos que gera, os URLs relativos e os formulários.

Elementos que geram pedidos automaticamente

Ao ler o HTML, o browser pede sozinho: `<link>` (CSS), `<script src>` (JS) e `<img>` (imagens). O `<a>` **não conta**, porque só gera um pedido quando clicas.

MNEMÓNICA · PEDIDOS AUTOMÁTICOS

«O browser é LIS: Link, Img, Script.»

Estes três pedem sem ninguém mandar. O **<a>** é educado: **espera o clique**.

URLs absolutos e relativos

```
Página: http://www.terrypratchettbooks.com/terry/index.html
<img src="/img/common/terry-big.jpg">  →  GET /img/common/terry-big.jpg        (começa por / = desde a raiz)
<img src="img/common/terry-big.jpg">   →  GET /terry/img/common/terry-big.jpg  (relativo à pasta da página)
<a href="talk.html">                     →  GET /terry/talk.html  (só ao clicar)
```

O elemento `<base href="...">` muda a base usada para resolver os URLs relativos.

MNEMÓNICA · BARRA INICIAL

«Com barra, vai à raiz. Sem barra, fica em casa (na pasta da página).»

Formulários

- `action` = para onde vai o pedido. `method` = GET (por omissão) ou POST.
- `enctype` = **como os dados do formulário são codificados** (gabarito). Valores: `application/x-www-form-urlencoded` (por omissão) ou `multipart/form-data` (obrigatório para enviar ficheiros).

> **enctype:** a resposta certa é «definir como os dados do formulário são codificados». «Enviar dados de tipos diferentes» só deu cotação parcial, e «cifrar» está errado (enctype vem de en**coding**, não de en**cryption**).

Helpers e plugins (já saíram no Moodle)

Quando o browser recebe um tipo MIME que não sabe mostrar:

	ONDE CORRE	DEFINIÇÃO DO GABARITO
Helper	**Fora** do browser	«aplicação **externa** que permite ao cliente web renderizar um conteúdo que ele desconhece»
Plugin	**Dentro** do browser	«aplicativo **executado dentro do cliente web** usado para renderizar conteúdos não conhecidos pelo próprio»

MNEMÓNICA · HELPER VS PLUGIN

«O Plugin está PLUGado lá dentro. O Helper é um ajudante que vem de fora.»

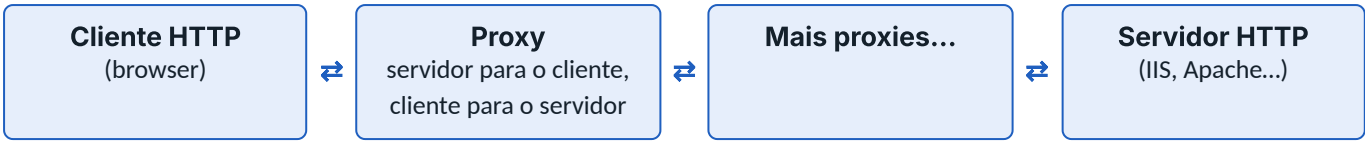
CAPÍTULO 11 · AULAS 02 E 06

Proxies e tipos de cache

Os intermediários entre o browser e o servidor, e os três sítios onde se guardam cópias das respostas.

O que é um proxy

Na prática, o browser e o servidor quase nunca falam diretamente: pelo meio há **um ou mais proxies**. O cliente faz o pedido ao proxy, o proxy faz o pedido ao servidor (ou ao proxy seguinte), recebe a resposta e devolve-a ao cliente.



Definição dos slides: os proxies atuam como **intermediários entre clientes e servidores** e podem ter vários propósitos. O seu uso pode **melhorar a segurança, o desempenho e a escalabilidade**, e oferece serviços como **cache**, **balanceamento de carga** (load balancing) e **filtragem de conteúdos**. O livro acrescenta que permitem passar mensagens HTTP **através de firewalls** e suportar **server farms** (várias máquinas que parecem um só servidor por fora).

MNEMÓNICA · O QUE FAZ UM PROXY

O proxy é o porteiro do prédio: Segura, Copia, Balança e Filtra.

Segura a porta (segurança, atravessar firewalls) · guarda **Cópias** das encomendas (cache) · **Balança** as visitas pelos elevadores (balanceamento de carga, server farms) · **Filtra** quem entra (filtragem de conteúdos).
E tem **duas caras**: para o cliente parece um servidor, para o servidor parece um cliente.

Ligações com outras matérias: o **502 Bad Gateway** é o proxy ou gateway a receber uma resposta inválida do servidor de cima (upstream). O **407 Proxy Authentication Required** é o «401 dos proxies». Um proxy que guardou um **301** em cache pode continuar a redirecionar mesmo depois de o servidor mudar (o problema do login).

Cache web: definição

Cache web é o **armazenamento temporário** de páginas, imagens e outros ficheiros, para **reduzir o uso da largura de banda** e **aumentar a velocidade de acesso**. Em vez de repetir o pedido à fonte original, usa-se a cópia guardada.

Os três tipos de cache

TIPO	ONDE FICA	O QUE EVITA	NOTAS
Server-side (lado do servidor)	No servidor	Evita que o servidor **gere a resposta outra vez do zero** (HTML, imagens geradas dinamicamente…)	Específica de cada aplicação, fora da norma HTTP. Exemplo: Output Caching no ASP.NET.
Client-side (lado do cliente, browser)	No browser (memória ou disco)	Evita que o cliente **faça o pedido**, ou que o servidor **reenvie** um recurso que não mudou	Controlada pelos cabeçalhos HTTP. Só serve um utilizador.
Proxy-side (nos proxies)	No proxy, entre cliente e servidor	Evita ir ao servidor de origem quando outro utilizador já pediu o mesmo	É uma **cache partilhada** por muitos utilizadores. Não pode guardar respostas `private`.

MNEMÓNICA · OS TRÊS TIPOS DE CACHE

Três frigoríficos: o do restaurante, o de casa e o do supermercado do bairro.

Servidor = frigorífico do restaurante: guarda pratos já feitos para o cozinheiro não cozinhar outra vez.
Cliente = frigorífico de casa: se já lá tens, nem sais de casa (nem fazes o pedido).
Proxy = supermercado do bairro: é **partilhado** pelos vizinhos, por isso não guarda coisas **privadas** de ninguém.

Como funciona a cache no cliente (browser)

Segundo Shklar & Rosen, a cache do browser precisa de três componentes:

- **Módulo de geração de pedidos**: antes de pedir, pergunta à cache se já existe uma cópia válida. Se a cópia expirou, acrescenta cabeçalhos de validação (`If-Modified-Since`).
- **Módulo de geração de respostas**: lê os cabeçalhos da resposta para saber se pode guardar e por quanto tempo.
- **Módulo de cache**: guarda os recursos e as datas de modificação (em memória ou no disco).

EXEMPLO DOS SLIDES	O QUE ACONTECE
1. Client caches a response	Primeiro pedido: `200 OK`, e o browser guarda a resposta.
2. Client cache hit	A cópia ainda é válida, por isso **nem há pedido** ao servidor.
3. Cached entry expires	A cópia passou o prazo (Expires ou max-age).
4. Revalidate hit	Pedido com `If-Modified-Since`. Não mudou, logo `304 Not Modified` sem corpo, e usa-se a cópia.
5. Revalidate miss	Mudou, logo `200 OK` com o conteúdo novo, que substitui a cópia.
6. Proxy cache miss	O proxy não tem a cópia: vai ao servidor, guarda a resposta e entrega-a.
7. Proxy cache hit	O proxy já tem a cópia (pedida antes, talvez por outro utilizador) e responde sem ir ao servidor.

Cabeçalhos que controlam a cache

Quem decide se uma resposta pode ir para a cache é o **servidor** ou a **aplicação**, porque são quem sabe se o conteúdo muda.

CABEÇALHO	PARA QUE SERVE
`Cache-Control: public`	Qualquer cache pode guardar, partilhada (proxy) ou local (browser).
`Cache-Control: private`	Só para um utilizador: o browser pode guardar, uma cache **partilhada** não.
`Cache-Control: no-cache`	Segundo o livro, **nem browsers nem proxies podem guardar** a resposta em cache. Na norma atual quer dizer «guardar, mas revalidar sempre antes de usar». No teste, segue o livro.
`Cache-Control: max-age=N`	Validade em segundos.
`Expires`	Data a partir da qual a cópia expira.
`Pragma: no-cache`	A versão **HTTP/1.0** (obsoleta). Envia-se junto com o Cache-Control para os agentes antigos.
`Last-Modified / If-Modified-Since`	Data da última alteração, e o pedido condicional que leva ao **304**.
`Date`	Quando a resposta foi gerada, usado para calcular a idade da cópia.

MNEMÓNICA · CACHE-CONTROL

Público, Privado, Proibido.

public = todos podem guardar · **private** = só o teu browser (nunca o proxy partilhado) · **no-cache** = ninguém guarda (versão do livro). O **Pragma** é o avô (HTTP/1.0): ainda se manda para os clientes antigos.

O que pode ir para cache: os métodos cacheáveis são sobretudo **GET** e **HEAD** (o POST normalmente não; lembra-te do «GET vs POST: GET pode ser guardado em cache»). Os códigos cacheáveis por omissão incluem `200`, `203`, `204`, `206`, `300`, `301`, `404`, `405`, `410`, `414` e `501`.

Configuração no servidor: no Apache faz-se com `mod_expires` e `mod_headers`, por extensão (`<FilesMatch "\.(gif|jpg|png|js|css)$"> Expires A2592000`) ou por tipo MIME (`ExpiresByType image/gif "access plus 1 month"`). No ASP.NET Core usa-se `[ResponseCache]` e **cache profiles** para não repetir as definições em cada action.

O que acrescenta o livro (Shklar & Rosen, pp. 172–182)

- **Cache expirada não é apagada.** Quando a entrada expira, fica guardada como «velha». No próximo pedido o browser revalida-a com `If-Modified-Since`. Se o servidor responde **304**, reutiliza a cópia e renova-lhe a validade.

Se responde **200**, substitui-a pelo conteúdo novo. O livro diz que deitar fora a cópia expirada «may not be necessary».

- **HTTP/1.0 vs HTTP/1.1.** No HTTP/1.0 o browser fazia um `HEAD`, comparava o `Last-Modified` e, se tivesse mudado, fazia um `GET`: podia precisar de **duas trocas**. No HTTP/1.1 basta **uma**, com `If-Modified-Since`.
- **Cache cheia.** O browser remove as entradas expiradas e depois as **menos usadas recentemente (LRU)**. Há cache em **RAM** e em **disco**, e as entradas podem passar da memória para o disco antes de serem apagadas. O tamanho máximo é configurável.
- **Chave da cache** = o **URL** pedido, incluindo query string e conteúdo POST. Um `Expires` com uma data que já passou faz com que a entrada só dure a sessão.
- **Anomalia do histórico.** O botão **Voltar** mostra muitas vezes a página guardada mesmo com `Cache-Control: no-cache`. O browser devia tratar o «Voltar» como um novo pedido.

MNEMÓNICA · CACHE EXPIRADA

logurte fora de prazo não desaparece do frigorífico.

Antes de o comeres, confirmas se ainda está bom (`If-Modified-Since`): se está, comes (**304**); se não, trocas por um novo (**200**). Só sai do frigorífico quando o trocas, quando o frigorífico fica cheio (LRU) ou quando o limpas. Atenção: um **cookie** expirado, pelo contrário, **é destruído**.

CAPÍTULO 12 · AULA 06

Autenticação e cookies

As outras duas «interações HTTP complexas»: provar quem és e lembrar-se de ti.

Autenticação HTTP

A **autenticação** verifica a **identidade** do utilizador. A **autorização** verifica se ele **tem acesso** a um recurso. A autenticação HTTP é feita com cabeçalhos: o servidor pede as credenciais e o cliente envia-as.

```
1. Cliente:  GET /privado HTTP/1.1
2. Servidor: HTTP/1.1 401 Unauthorized
             WWW-Authenticate: Basic realm="User Visible Realm"     ← <type> realm=<área
protegida>
3. Cliente:  GET /privado HTTP/1.1
             Authorization: Basic QWxhZGRpbjpPcGVuU2VzYW1l           ← utilizador:password em
Base64
4. Servidor: HTTP/1.1 200 OK   (e o browser repete o Authorization nos pedidos seguintes do
mesmo realm)
```

- **Esquemas**: Basic, Bearer, Digest, HOBA, Mutual, Negotiate, OAuth, SCRAM e vapid.
- O Basic só **codifica** em Base64, **não cifra**, por isso deve ir sobre HTTPS.
- Quando é um **proxy** a pedir autenticação, a resposta é **407 Proxy Authentication Required**.

MNEMÓNICA · AUTENTICAÇÃO

«Autenticação = quem és. Autorização = o que podes.»

O servidor responde **401 + WWW-Authenticate** («identifica-te!»). O cliente responde com **Authorization** («aqui está o meu cartão»). O 407 é o mesmo, mas quem pede o cartão é o porteiro do meio (o proxy).

Cookies

São pares **nome=valor** que o servidor pede ao browser para guardar e devolver. São a forma de manter estado sobre um HTTP **stateless** (login, carrinho, preferências).

```
Resposta do servidor:  Set-Cookie: Name=Leon; Domain=.rutgers.edu; Path=/test/
Pedidos seguintes:     Cookie: Name=Leon      (só para *.rutgers.edu com caminho a começar por
/test/)
```

TIPO	CARACTERÍSTICA
De sessão	Só em memória, **sem data de expiração**, apagado ao **fechar o browser**.
Persistente	Guardado no dispositivo, **com data de expiração** definida pelo servidor. Sobrevive ao fecho do browser.
First-party	Criado pelo **domínio que estás a visitar** (língua, analytics, preferências).
Third-party	Criado por **outros domínios**: rastreio entre sites, retargeting, publicidade (tracking cookies).

Estrutura: nome · valor · data de expiração (Expires ou Max-Age) · path · domain · Secure (só por ligação segura) · HttpOnly (só por HTTP, sem acesso por scripts).

MNEMÓNICAS · COOKIES

Set-Cookie vs Cookie: «O servidor dá-te a **senha do talho** (Set-Cookie). Tu mostras a senha em cada visita (Cookie).»
Sessão vs persistente: «A senha do talho deita-se fora ao sair (sessão). O cartão de cliente tem validade e fica na carteira (persistente).»
First vs third-party: «O cartão da loja onde estás vs o espião da publicidade colado ao teu casaco.»
Estrutura: «**N**unca **V**i **E**lefantes **P**assar **D**epressa **S**em **H**esitar»: Nome, Valor, Expiração, Path, Domain, Secure, HttpOnly.

Variáveis de sessão (exame de 2021): os dados ficam **no servidor** e o cookie leva só o **identificador da sessão**. Sem cookies (cookieless), o identificador vai na **query string** ou como **parte do URL**.

Quem usa cada cabeçalho

CABEÇALHO	AGENTE QUE O ENVIA	MENSAGEM
`Set-Cookie`	**Servidor** (servidor web ou aplicação)	Resposta
`Cookie`	**Browser**	Pedido
`WWW-Authenticate`	**Servidor**	Resposta (com 401)
`Authorization`	**Browser**	Pedido

Regras de domain e path (livro, 7.6)

- `domain=.mysite.com` aplica-se a `ecommerce.mysite.com` e `toys.ecommerce.mysite.com`, mas **não** a `mysite.com`.
- Um site **só pode definir cookies para o seu próprio domínio**: `ecommerce.mysite.com` pode usar `.mysite.com`, mas nunca `.yahoo.com`. Por isso os cookies de terceiros só entram através de **recursos de apoio** (imagens, CSS, JS, anúncios) servidos por outros domínios.
- `path=/` faz o cookie seguir em todos os pedidos ao servidor. `path=/cgi-bin/` só nos URLs que começam por `/cgi-bin/`.
- Os cookies criam uma **sessão lógica**, não uma sessão de rede.

Privacidade e P3P (livro, 7.7)

Os **cookies de terceiros** são considerados «particularmente perigosos» porque permitem seguir o utilizador entre sites. O **P3P** (Platform for Privacy Preferences, do W3C) permite ao site declarar a sua **política de privacidade**, legível por humanos e por máquinas, num `<LINK>`, num cabeçalho de resposta `P3P` ou em `/w3c/p3p.xml`. O browser compara-a com as preferências do utilizador e pode **aceitar**, **bloquear**, **despromover** (tornar não persistente) ou fazer **leashing** dos cookies (só os envia a terceiros quando o pedido vem de uma página do site original). Funciona «por honra»: o browser não consegue verificar se a política é cumprida.

MNEMÓNICA · O QUE O P3P DEIXA O BROWSER FAZER

«Aceita, Bloqueia, Despromove ou põe-lhe a Trela.»

Leash quer dizer trela: o cookie de terceiros só sai à rua acompanhado pelo site original.

Autorização: o que acrescenta o livro (7.8.2)

- Os servidores protegem recursos por **gamas de IP** e por uma **ACL** (Access Control List, com utilizadores e passwords), organizada em **realms**.
- Credenciais erradas levam a **401 outra vez**. Ao fim de algumas tentativas, a maioria dos servidores passa a **403 Forbidden**.
- As credenciais aceites são reutilizadas para o mesmo realm.

Persistência no browser: cache, cookies e credenciais (tabela 7.2)

	CACHE	COOKIES	CREDENCIAIS
Guarda?	Conforme os cabeçalhos da resposta	Se o utilizador aceita cookies e a política (P3P) o permite	Sempre
Chave	URL (com query e POST)	Domain + path	Servidor + realm
Validade	`Expires`. Data já passada: só a sessão. Cache cheia: LRU	Sem data: só a sessão. **Data já passada: apagado**	**Só a sessão, só em memória, nunca em disco**

MNEMÓNICA · CHAVES DE PERSISTÊNCIA

«A cache lembra-se do endereço (URL), o cookie da rua e da porta (domain e path), as credenciais do prédio e do andar (servidor e realm).» E as credenciais são como uma pulseira de festival: valem só enquanto dura a festa (sessão) e nunca vão para casa (disco).

Helpers e plugins (livro, 7.8.5)

Para formatos que não conhece, o browser tem 3 opções: **suporte nativo**, **helper** ou **plugin**. O **helper** apresenta o objeto **fora da janela** do browser (confuso para o utilizador) e só funciona com **links diretos** para o objeto. O **plugin** mostra-o **dentro da página**, através de `<EMBED>` ou `<OBJECT>` (o `<EMBED>` está obsoleto em favor de `<OBJECT>`). Se não houver associação para um tipo MIME, o browser usa um helper e deixa o sistema operativo escolher. É o **módulo de interpretação de conteúdo** que analisa o HTML e pede ao módulo de geração de pedidos os recursos de apoio.

CAPÍTULO 13

Armadilhas do gabarito do professor

O que aprendemos com as perguntas reais do Moodle. Lê isto na véspera.

1. **Muitas perguntas têm várias respostas certas, mesmo com aspeto de escolha única.** No Moodle, cada opção errada desconta. Assinala todas as que são verdade, e só essas.
2. **Prefere a opção mais completa.** «HTTP é um protocolo de comunicação» deu meia cotação; a completa era «usado por todos os agentes web num canal de comunicação». O mesmo com o Plugin: «executado dentro do cliente web *usado para renderizar conteúdos não conhecidos*».
3. **«Nenhuma das opções» pode ser a certa** (PHP/Java/Perl). Se todas as opções têm um erro («unicamente», «estáticos», «armazenamento», «pelos clientes»), escolhe-a.
4. **Palavras absolutas** como «apenas», «unicamente» e «sempre» costumam tornar a opção falsa.
5. **O servidor nunca faz pedidos ao cliente.**
6. **Para o cliente, dinâmico é igual a estático.**
7. **404**: o gabarito aceitou «erro de processamento do pedido» e «pedido mal formulado».
8. **Códigos de estado servem para** classificar a resposta com tipo e contexto, e para o cliente a interpretar depressa (2 certas).
9. **Linha de pedido**: `GET / HTTP/1.1`, sem domínio e sem «HTTPS/1.1».
10. **Elementos que pedem sozinhos**: `<link>`, `<script>` e `<img>`, sem o `<a>`.
11. **Aplicação web** precisa de um servidor e um cliente.
12. **Header HTTP**: usa-se nos pedidos e nas respostas.

Resumo das respostas de várias opções certas

PERGUNTA	RESPOSTAS CERTAS (GABARITO)
Referer	URL da página mostrada · local de origem do pedido · referenciar recurso noutro servidor · distinguir clique de URL escrito
Accept-Encoding	Cliente aceita conteúdos comprimidos · método de compressão a usar
image/gif	Corpo tem os bytes de uma imagem · o cliente recebeu uma imagem
502 Bad Gateway	Proxy/gateway não obtém bom resultado do upstream · falha entre servidores que retransmitem dados
404	Erro de processamento do pedido · pedido mal formulado
Códigos de estado	Classificar com tipo e contexto · interpretação fácil e rápida
Pseudónimos	Mascarar caminhos · mudar nomes · associar recursos externos
Conteúdos estáticos	Conteúdo estático · corresponde integralmente à resposta · sem processar código
Autenticação	Verifica permissões de acesso · gera respostas 401

Estratégia para o dia 22

- Lê todas as opções antes de marcar. As do professor são parecidas e diferem numa palavra.
- Em cada opção pergunta-te: «isto é **verdade** sobre o conceito?». Se for, marca.
- Se ficares entre duas opções, escolhe a que usa as palavras dos slides (por exemplo «agentes web», «upstream», «permissões de acesso»).
- Faz os testes-modelo na página de treino até teres 17 ou mais em todos.

CAPÍTULO 14

Cábula de uma página

Todas as mnemónicas juntas. Relê isto na manhã do teste.

CLIENTE-SERVIDOR

Freguês e empregado: o servidor **nunca** pede ao cliente. Uma app web precisa de **um servidor e um cliente**.

WEB VS DESKTOP

MAIS: Multiplataforma, Atualizações, Instalação dispensada, Sem gastar recursos. **DOS**: Desempenho, Offline, Segurança.

PORTAS

FTP 20/21 · SSH 22 · Telnet 23 · SMTP 25 · DNS 53 · **HTTP 80** · POP3 110 · IMAP 143 · **HTTPS 443**

URL

«Eu Uso Senha Num Porto, Passo Pela Query, Até à Âncora». O espaço passa a `%20` ou `+`.

PEDIDOS DE UMA PÁGINA

1 HTML + 1 por imagem, CSS e script, possivelmente a outros servidores. O JS pode gerar mais.

ESTÁTICO VS DINÂMICO

Ao cliente chega o prato, não a receita: para o browser é igual.

HTTP

Aplicação, sobre TCP, pedido-resposta. **Stateless**: peixinho dourado. Proxy = servidor para o cliente e cliente para o servidor (cache, filtro, balanceamento).

MENSAGENS

Pedido **MUV**: Método URI Versão. Resposta **VCF**: Versão Código Frase. Depois: cabeçalhos, linha vazia (CRLF), corpo.

MÉTODOS

0.9 GET · 1.0 GHP (+HEAD, POST) · 1.1 o resto. REST: POST cria, GET lê, PUT substitui, PATCH remenda, DELETE apaga.

GET VS POST

GET = postal: URL visível, limitado, ASCII, favoritos, cache, histórico. POST = carta fechada: corpo, sem limite, binário, multipart.

GRUPOS DE CABEÇALHOS

«Gatos Raramente Respondem Educadamente»: General, Request, Response, Entity. **Content-*** = Entity.

CABEÇALHOS-CHAVE

Host (virtual hosting) · User-Agent · Referer · Authorization · Accept-Encoding (compressão) · If-Modified-Since → 304 · Location (301/302) · WWW-Authenticate (401) · Content-Type (MIME) · Content-Length (bytes)

PROIBIDOS

Quem sou (Host, Origin, Referer, Cookie) · por onde vou (Connection, Keep-Alive, Content-Length, Date) · o que aceito (Accept-Charset, Accept-Encoding) · Proxy-*, Sec-*

MIME

Família/Pessoa; Detalhe: `text/html; charset=utf-8`. Charset por omissão ISO-8859-1. multipart + boundary. `...` evita um pedido.

CLASSES DE ESTADO

Aguenta (1) · Toma (2) · Vai (3) · Tu (4) · Eu (5)

CÓDIGOS

301 morada nova · 302 casa de férias (RedirectToAction) · 304 usa a cache · 400 mal formulado · 401 quem és tu? · 403 sei quem és mas não entras · 404 não existe · 500 rebentou · 502 o estafeta trouxe lixo · 503 em baixo

SERVIDOR

RPG: Resolve, Processa, Gera. Resolução de endereços: «Vê A Pasta Autorizada» (Virtual host, Address mapping, Pseudónimos, Autenticação).

RAÍZES WEB

Apache em Linux `/var/www` · IIS `c:\Inetpub\wwwroot` · ASP.NET Core `wwwroot`. Pseudónimo = nome artístico: mascara, renomeia, traz de fora.

AUTENTICAÇÃO

401 + WWW-Authenticate, depois Authorization: Basic (Base64). O módulo verifica permissões e gera os 401.

ESTÁTICO

Static: o servidor põe o envelope. As-is: a carta já vem no envelope.

DINÂMICO

«Cada Servidor Ainda Faz Truques Super Java»: CGI, SSI, APIs nativas (ISAPI, Apache API), FastCGI, Templates (PHP, ColdFusion, ASP), Servlets, JSP. CGI = um táxi por passageiro.

HTML NO CLIENTE

LIS pede sozinho (link, img, script), o <a> espera o clique. Com barra vai à raiz, sem barra fica na pasta. enctype = codificação.

HELPER VS PLUGIN

Helper = ajudante de fora (externo). Plugin = plugado dentro do browser.

PROXY

Porteiro com duas caras (servidor para o cliente, cliente para o servidor): Segura, Copia, Balança, Filtra. 502 e 407 envolvem proxies.

TIPOS DE CACHE

Três frigoríficos: servidor (não gerar outra vez), cliente/browser (não pedir), proxy (partilhado, nunca guarda private). Revalidação: If-Modified-Since, depois 304.

CACHE-CONTROL

Público, Privado, Proibido (public, private, no-cache) + max-age · Expires · Pragma: no-cache (HTTP/1.0) · cacheáveis: GET e HEAD

LIVRO PP. 172–182

Cache expirada não é apagada (iogurte). Set-Cookie = servidor, Cookie = browser. Domain: só o próprio domínio. P3P: aceita, bloqueia, despromove, trela. 401 repetido → 403. Chaves: URL / domain+path / servidor+realm. Credenciais só em memória.

AUTENTICAÇÃO E COOKIES

Quem és (autenticação) vs o que podes (autorização). Set-Cookie (servidor) e Cookie (browser). Sessão = senha do talho, persistente = cartão com validade. NVEPDSH.